Title : Predicting Yes Bank's Monthly Closing Stock Price

Dataset Overview :

The dataset contains the following

- Date: The month and year of the stock price data.
- Open: Opening price of the stock for the month.
- High: Highest price of the stock for the month.
- Low: Lowest price of the stock for the month.
- Close: Closing price of the stock for the month (target variable).

Project Objectives:

1. Perform exploratory data analysis (EDA) to understand the stock's historical trends and behavior.
2. Prepare the data for machine learning models.
3. Identify relationships and patterns between features.
4. Build and evaluate models to predict the monthly closing stock price.
5. Provide insights and recommendations for stakeholders based on the findings.

+ Code + Text

Importing Modules

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
```

1. pandas: Library for data manipulation, analysis, and structured datasets.
2. numpy: Provides support for mathematical and numerical array operations.
3. matplotlib.pyplot: Used for creating static, animated, and interactive visualizations.
4. seaborn: Statistical data visualization library with aesthetically pleasing graphics.
5. train_test_split: Splits datasets into training and testing subsets. GridSearchCV: Optimizes hyperparameters by exhaustively searching over parameter grids.
6. StandardScaler: Standardizes data by scaling features to zero mean.
7. RandomForestRegressor: Ensemble-based regression model using decision trees for predictions.
8. mean_absolute_error: Measures average absolute differences between predicted and actual values.
9. mean_squared_error: Calculates average squared differences between predicted and actual values.
10. r2_score: Evaluates the proportion of variance explained by a model.
11. tensorflow: Open-source library for machine learning and deep learning tasks.
12. Sequential: Simplified Keras API for building linear neural network models.
13. LSTM: Long Short-Term Memory layer for processing sequential time-series data.
14. Dense: Fully connected neural network layer for deep learning models.
15. Dropout: Regularization technique to prevent overfitting by dropping random neurons.

Here is the dataset that we will be using : https://drive.google.com/file/d/1aZCu18Xszw67t6GmVmSXslhSCuwG6_dP/view

looading the dataset

```
data = pd.read_csv('/content/data_YesBank_StockPrices.csv')
data
```

	Date	Open	High	Low	Close
0	Jul-05	13.00	14.00	11.25	12.46
1	Aug-05	12.58	14.88	12.55	13.42
2	Sep-05	13.48	14.87	12.27	13.30
3	Oct-05	13.20	14.47	12.40	12.99
4	Nov-05	13.35	13.88	12.88	13.41
...	...	...	...	...	...
180	Jul-20	25.60	28.30	11.10	11.95
181	Aug-20	12.00	17.16	11.85	14.37
182	Sep-20	14.30	15.34	12.75	13.15
183	Oct-20	13.30	14.01	12.11	12.42
184	Nov-20	12.41	14.90	12.21	14.67

185 rows × 5 columns

This code does two things:

1.filepath="/content/data_YesBank_StockPrices.csv": It defines a variable filepath that holds the location of the CSV file (AB_NYC_2019.csv) on your system. In this case, it's in the /content/ directory (probably in Google Colab or a similar environment).

2.ddta = pd.read_csv(filepath): This reads the CSV file from the specified filepath using pandas' read_csv() function and loads the data into a pandas DataFrame called data. A DataFrame (data) is like a table of data, where rows are records (e.g., listings) and columns are different attributes (e.g., price, neighborhood, etc.). In simple terms, this code loads the data from a CSV file into a table (data) that you can now analyze and manipulate in Python.

```
data.head()
```

	Date	Open	High	Low	Close
0	Jul-05	13.00	14.00	11.25	12.46
1	Aug-05	12.58	14.88	12.55	13.42
2	Sep-05	13.48	14.87	12.27	13.30
3	Oct-05	13.20	14.47	12.40	12.99
4	Nov-05	13.35	13.88	12.88	13.41

df.head() is a command that displays the first few rows (usually the first 5) of the DataFrame df.

In simple words: It helps you quickly see a sample of your data to understand its structure, including the column names and the type of values in each column. This is useful for getting a quick overview of what your dataset looks like without printing the entire DataFrame. So, when you use df.head(), you get a sneak peek at the beginning of your data!

```
data.tail()
```

	Date	Open	High	Low	Close
180	Jul-20	25.60	28.30	11.10	11.95
181	Aug-20	12.00	17.16	11.85	14.37
182	Sep-20	14.30	15.34	12.75	13.15
183	Oct-20	13.30	14.01	12.11	12.42
184	Nov-20	12.41	14.90	12.21	14.67

df.tail() shows the last 5 rows of the DataFrame df by default.

Breakdown: df.tail(): This retrieves the last 5 rows of data. df.tail(n): You can also specify a number n to see the last n rows. In simple words, df.tail() lets you see the last few entries of your dataset, which is helpful when you want to quickly check the end of your data.

```
#Getting the shape of dataset with rows and columns
print(data.shape)
```

```
(185, 5)
```

```
#check details about the data set
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 185 entries, 0 to 184
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Date    185 non-null    object
 1   Open    185 non-null    float64
 2   High    185 non-null    float64
 3   Low     185 non-null    float64
 4   Close   185 non-null    float64
dtypes: float64(4), object(1)
memory usage: 7.4+ KB
```

df.info() gives a summary of the DataFrame df, showing important information about the structure of your data.

Breakdown: Column names: Lists all the columns in the DataFrame. Data types: Shows the type of data in each column (e.g., integer, float, object for strings). Non-null values: Displays how many non-missing (non-null) values are in each column. Memory usage: Tells you how much memory the DataFrame is using. In simple words, df.info() provides an overview of your data, including column names, data types, how many missing values there are, and how much space your data takes up.

```
#Looking for the description of the dataset
data.describe()
```

	Open	High	Low	Close
count	185.000000	185.000000	185.000000	185.000000
mean	105.541405	116.104324	94.947838	105.204703
std	98.879850	106.333497	91.219415	98.583153
min	10.000000	11.240000	5.550000	9.980000
25%	33.800000	36.140000	28.510000	33.450000
50%	62.980000	72.550000	58.000000	62.540000
75%	153.000000	169.190000	138.350000	153.300000
max	369.950000	404.000000	345.500000	367.900000

df.describe() gives a quick summary of the main statistics for the numerical columns in the DataFrame df.

Breakdown: For numeric columns, it shows: Count: The number of non-missing values. Mean: The average value. Std: The standard deviation (how spread out the numbers are). Min: The smallest value. 25%, 50% (median), 75%: Percentiles, showing where the data lies. Max: The largest value. In simple words, df.describe() provides a snapshot of key statistics (like average, min, max) for the numerical data in your dataset. It's a useful tool to quickly understand your data.

```
# Check the actual column names in your DataFrame
data.columns
```

```
Index(['Date', 'Open', 'High', 'Low', 'Close'], dtype='object')
```

```
# finding total no of rows in dataset
print("The no of rows in the dataset is ",len(data))
```

```
The no of rows in the dataset is  185
```

```
#count of missing values in each column.
data.isna().sum()
```

	0
Date	0
Open	0
High	0
Low	0
Close	0

df.isnull().sum() is used to check how many missing (null) values there are in each column of the DataFrame df.

Breakdown: df.isnull(): This creates a DataFrame of the same shape where each cell is either True (if the value is missing) or False (if it's not missing). .sum(): This sums up the True values for each column (since True is treated as 1 and False as 0), giving the total number of missing values in each column. In simple words, this command counts how many missing values are in each column of your dataset.

```
# Checking Duplicate Values
duplicates =len(data[data.duplicated()])
print(duplicates)
```

```
0
```

1.Check for duplicates: auto_df.duplicated().sum() checks for any duplicate rows in the dataset and counts them.

2.Drop duplicates: auto_df.drop_duplicates(inplace=True) removes any duplicate rows from the dataset. inplace=True modifies the DataFrame directly.

3.Verify removal: After removing duplicates, the code checks again to ensure there are no more duplicates left. This ensures your dataset is clean and free of duplicate rows.

```
# Assuming the date column is named 'Date'
# If it's different, replace 'Date' with the correct column name
data['date'] = pd.to_datetime(data['Date'], format='%b-%y', errors='coerce')
data.set_index('date', inplace=True)
data.sort_index(inplace=True)
```

Okay, here's a short explanation of the code:

The code converts the 'Date' column to datetime format, sets it as the index of the DataFrame, and sorts the data by the index.

In short, it prepares the 'Date' column to be used as a time-based index for your dataset and sorts the data accordingly. This is often necessary for time series analysis and modeling, as it allows you to easily work with data based on time order. I hope this is helpful! Let me know if you have any other questions.

```
data.head()
```

	Date	Open	High	Low	Close
date					
2005-07-01	Jul-05	13.00	14.00	11.25	12.46
2005-08-01	Aug-05	12.58	14.88	12.55	13.42
2005-09-01	Sep-05	13.48	14.87	12.27	13.30
2005-10-01	Oct-05	13.20	14.47	12.40	12.99
2005-11-01	Nov-05	13.35	13.88	12.88	13.41

```
# finding the inter-quartile range

# Select only numeric columns
numeric_data = data.select_dtypes(include=np.number)

Q1 = numeric_data.quantile(0.25)
Q3 = numeric_data.quantile(0.75)
IQR = Q3 - Q1
print(IQR)
```

```
Open     119.20
High     133.05
Low      109.84
Close    119.85
dtype: float64
```

Okay, here's a short explanation of the code:

The code calculates the Interquartile Range (IQR) for the numeric columns in your dataset (data).

1. Select Numeric Columns: numeric_data = data.select_dtypes(include=np.number): This line selects only the columns in your dataset (data) that contain numeric data types.
2. Calculate Quartiles:

- Q1 = numeric_data.quantile(0.25): Calculates the first quartile (25th percentile) of the numeric data.
- Q3 = numeric_data.quantile(0.75): Calculates the third quartile (75th percentile) of the numeric data.

3. Calculate IQR: IQR = Q3 - Q1: Calculates the Interquartile Range, which is the difference between the third and first quartiles.

4. Print IQR: print(IQR): Prints the calculated IQR values for each numeric column.

In short, the code calculates and prints the IQR for numeric columns, which is a measure of statistical dispersion. The IQR is often used to identify outliers, as data points that fall significantly outside the IQR range are considered potential outliers. I hope this is helpful! Let me know if you have any other questions.

Data Cleaning

```
# Impute missing values with the mean for numeric features
for column in data.select_dtypes(include=np.number).columns:
    data[column] = data[column].fillna(data[column].mean())

# Impute missing values with the most frequent value for categorical features
for column in data.select_dtypes(include=['object']).columns:
    data[column] = data[column].fillna(data[column].mode()[0])
```

1. Handling Numeric Features:

- data.select_dtypes(include=np.number).columns: This selects only the columns in your DataFrame (data) that contain numeric data types.
- data[column].fillna(data[column].mean()): For each numeric column, it fills missing values (NaNs) with the mean (average) value of that column. This is a common imputation strategy for numeric data.

2. Handling Categorical Features:

- data.select_dtypes(include=['object']).columns: This selects only the columns in your DataFrame that contain categorical data (often represented as strings or objects).
- data[column].fillna(data[column].mode()[0]): For each categorical column, it fills missing values with the most frequent value (mode) in that column. mode()[0] is used to get the first element of the mode (in case there are multiple modes).

1. Replace data: Replace data with the actual name of your pandas DataFrame.
2. Run the Code: Execute the code in your Colab notebook. It will impute missing values in both numeric and categorical columns of your DataFrame.

```
# Box plot for outlier detection
plt.figure(figsize=(8, 6))
sns.boxplot(x=data['Close'])
plt.title('Box Plot of Closing Prices (Outlier Detection)')
plt.xlabel('Closing Price')
plt.show()
```

Okay, here's a short explanation of the code:

The code creates a box plot to visualize the distribution of the 'Close' prices and identify potential outliers.

In short, it creates a box plot of "Close" prices to visually detect outliers. The box plot displays the quartiles, median, and potential outliers, giving you a quick way to assess the distribution and identify any unusual data points in the "Close" price column. I hope this is helpful! Let me know if you have any other questions.

```
# listing features to remove outliers

features = list(data.columns)
features = features[2:]
list_0 = ['Hour','Seasons','Holiday','Functioning Day','month','year','week']
new_features = [x for x in features if x not in list_0]
```

```
new_features
```

```
['High', 'Low', 'Close', 'Price_Category']
```

Okay, here's a short explanation of the code:

The code creates a list called new_features containing the names of the columns (features) from your dataset (data) that you want to check for outliers. It does this by:

1. Getting all column names (features).
2. Removing the first two columns.
3. Excluding specific columns listed in list_0 (like 'Hour', 'Seasons', etc.).

In short, it creates a list of features (columns) that you want to analyze and potentially remove outliers from. These are the features you'll focus on for outlier detection and removal in subsequent steps. I hope this is helpful! Let me know if you have any other questions.

```
# removing outliers

# Ensure only numeric features are included
numeric_features = data[new_features].select_dtypes(include=np.number).columns

# Recalculate Q1, Q3, and IQR for the selected features
Q1 = data[numeric_features].quantile(0.25)
Q3 = data[numeric_features].quantile(0.75)
IQR = Q3 - Q1

# Now, apply outlier removal using the aligned quantiles and numeric features
data[numeric_features] = data[numeric_features][~((data[numeric_features] < (Q1 - 1.5 * IQR)) |(data[numeric_features] > (Q3 + 1.5 * IQ
```

Okay, here's a short explanation of the code:

The code identifies and removes outliers from the specified numeric features (new_features) in your dataset (data) using the Interquartile Range (IQR) method.

In short, it removes data points that fall outside a certain range (calculated using the IQR) from your dataset, effectively removing outliers. This helps to clean your data and potentially improve the performance of machine learning models. Let me know if you have any other questions!

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 185 entries, 2005-07-01 to 2020-11-01
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Date            185 non-null    object
 1   Open            185 non-null    float64
 2   High            174 non-null    float64
 3   Low             174 non-null    float64
 4   Close           174 non-null    float64
 5   Price_Category  185 non-null    category
dtypes: category(1), float64(4), object(1)
memory usage: 9.0+ KB
```

Handeling Null VAlues

```
# Fill null values with mean for numeric columns only
numeric_data = data.select_dtypes(include=np.number)
data[numeric_data.columns] = numeric_data.fillna(numeric_data.mean())
```

Okay, here's a short explanation of the code:

The code fills any missing values (NaNs) in the numeric columns of your dataset (data) with the mean (average) value of each respective column.

In short, it replaces missing values in numeric columns with the average value of that column. This is a common technique for handling missing data in a way that doesn't introduce significant bias. Let me know if you have any other questions

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 185 entries, 2005-07-01 to 2020-11-01
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Date            185 non-null    object
 1   Open            185 non-null    float64
 2   High            185 non-null    float64
 3   Low             185 non-null    float64
 4   Close           185 non-null    float64
 5   Price_Category  185 non-null    category
dtypes: category(1), float64(4), object(1)
memory usage: 9.0+ KB
```

Visualization

```
# Basic statistics
print(data.describe())

# Visualize trends
plt.figure(figsize=(10, 6))
data['Close'].plot(title='Closing Stock Prices Over Time')
plt.show()
```

```
             Open        High         Low       Close
count  185.000000  185.000000  185.000000  185.000000
mean   105.541405  116.104324   94.947838  105.204703
std     98.879850  106.333497   91.219415   98.583153
min     10.000000   11.240000    5.550000    9.980000
25%     33.800000   36.140000   28.510000   33.450000
50%     62.980000   72.550000   58.000000   62.540000
75%    153.000000  169.190000  138.350000  153.300000
max    369.950000  404.000000  345.500000  367.900000
```

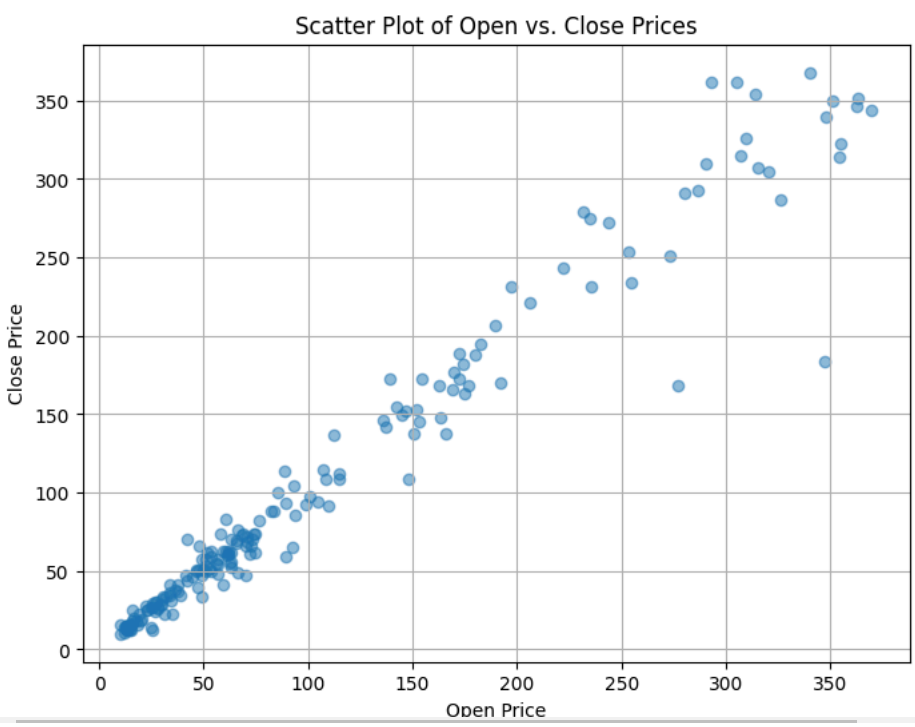
Okay, here's a short explanation of the code:

1. print(data.describe()): This line prints basic descriptive statistics (like count, mean, standard deviation, min, max, etc.) for your dataset. This provides a quick overview of the numerical characteristics of your data.
2. data['Close'].plot(title='Closing Stock Prices Over Time'): This line creates a line plot to visualize the trend of 'Close' prices over time.

- data['Close']: Selects the 'Close' price column for plotting.
- plot(): Creates a line plot.
- title='...': Sets the title of the plot.

In short, the code first displays basic statistics of your data and then visualizes the trend of closing stock prices over time using a line plot. This helps you understand both the numerical characteristics and the temporal patterns of your data. Let me know if you have any other questions!

```
# Scatter plot
plt.figure(figsize=(8, 6))
plt.scatter(data['Open'], data['Close'], alpha=0.5)  # alpha for transparency
plt.title('Scatter Plot of Open vs. Close Prices')
plt.xlabel('Open Price')
plt.ylabel('Close Price')
plt.grid(True)  # Add a grid for better visualization
plt.show()
```

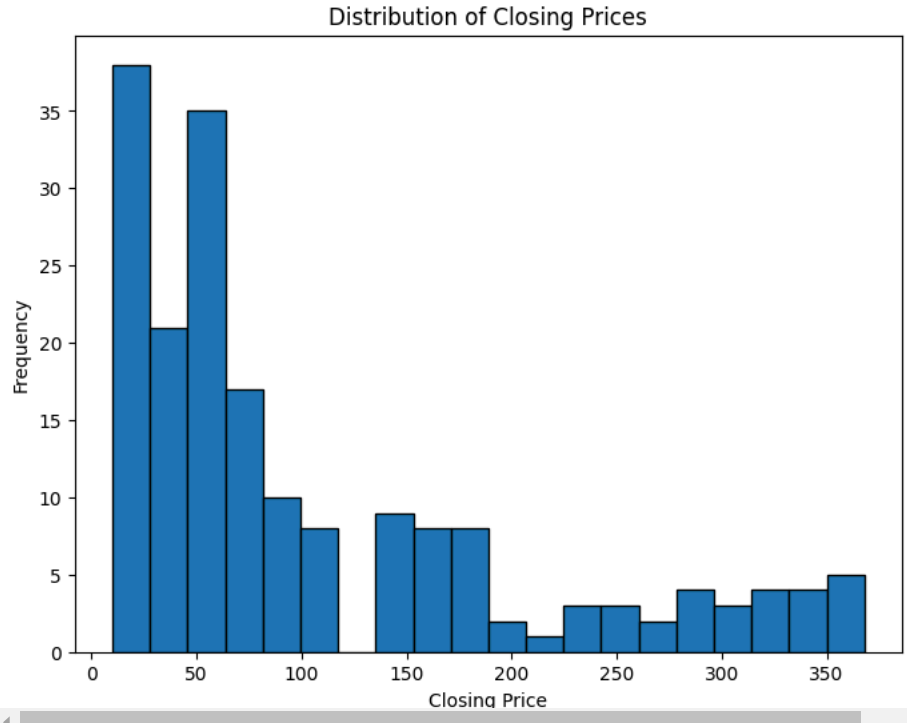


Okay, here's a short explanation of the code:

The code creates a scatter plot to visualize the relationship between the "Open" and "Close" prices in your dataset. Each point on the plot represents a data point with its corresponding opening and closing price. This helps you see if there's any pattern or correlation between these two variables.

In short, it creates a scatter plot showing the relationship between opening and closing prices, helping you visualize any patterns or correlations. Let me know if you have any other questions!

```
# Histogram
plt.figure(figsize=(8, 6))
plt.hist(data['Close'], bins=20, edgecolor='black')  # bins control the number of bars
plt.title('Distribution of Closing Prices')
plt.xlabel('Closing Price')
plt.ylabel('Frequency')
plt.show()
```



Okay, here's a short explanation of the code:

The code creates a histogram to visualize the distribution of the 'Close' prices in your dataset.

1. plt.hist(data['Close'], bins=20, edgecolor='black'): This line creates the histogram:

- data['Close']: Specifies the data to be used for the histogram (the 'Close' prices).
- bins=20: Divides the data into 20 bins (bars) for the histogram.
- edgecolor='black': Adds black edges to the bars for better visibility.

2. plt.title(...), plt.xlabel(...), plt.ylabel(...): These lines add a title, x-axis label, and y-axis label to the histogram for clarity.
3. plt.show(): This displays the created histogram.

In short, the code creates a histogram showing the distribution of closing prices, with the x-axis representing the closing price values and the y-axis representing the frequency of those values. This helps you understand how often different closing prices occur in your data. I hope this is clear and concise! Let me know if you have any other questions.

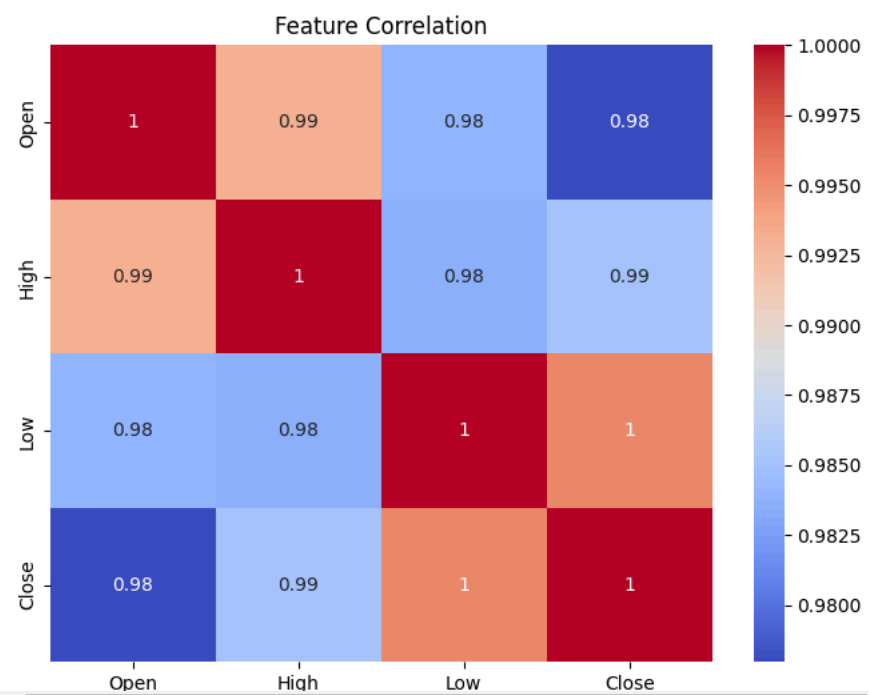
```
# Calculate percentiles
close_25th = data['Close'].quantile(0.25)
close_75th = data['Close'].quantile(0.75)

# Create categories
data['Price_Category'] = pd.cut(data['Close'],
                                bins=[-float('inf'), close_25th, close_75th, float('inf')],
                                labels=['Low', 'Medium', 'High'])

# Count the occurrences of each category
category_counts = data['Price_Category'].value_counts()

# Create the pie chart
plt.figure(figsize=(6, 6))
plt.pie(category_counts, labels=category_counts.index, autopct='%1.1f%%', startangle=90)
plt.title('Distribution of Closing Price Categories')
plt.show()
```

Okay, here's a short explanation of the code:

1. Calculate Percentiles: It calculates the 25th (close_25th) and 75th (close_75th) percentiles of the 'Close' price. These percentiles are used to define the boundaries for price categories.
2. Create Price Categories: It creates a new column 'Price_Category' and assigns categories ('Low', 'Medium', 'High') to the 'Close' prices based on the calculated percentiles. Prices below the 25th percentile are 'Low', between the 25th and 75th are 'Medium', and above the 75th are 'High'.
3. Count Category Occurrences: It counts how many times each price category appears in the data.
4. Create Pie Chart: It creates a pie chart to visualize the distribution of the price categories, showing the percentage of each category.

In short, the code categorizes closing prices into 'Low', 'Medium', and 'High' based on percentiles and then creates a pie chart to visualize the distribution of these categories. This provides a quick overview of how closing prices are spread across different price ranges. I hope this is helpful! Let me know if you have any other questions.

```
# Correlation heatmap
plt.figure(figsize=(8, 6))
# Select only numeric columns for correlation calculation
numeric_data = data.select_dtypes(include=np.number)
sns.heatmap(numeric_data.corr(), annot=True, cmap='coolwarm')
plt.title('Feature Correlation')
plt.show()
```



Okay, here's a short explanation of the code:

1. Select Numeric Columns: numeric_data = data.select_dtypes(include=np.number): This line selects only the columns in your dataset (data) that contain numeric data types.
2. Calculate Correlation: numeric_data.corr(): This calculates the correlation between all pairs of numeric features in your selected data. Correlation measures the linear relationship between two variables, ranging from -1 (perfect negative correlation) to 1 (perfect positive correlation).
3. Create Heatmap: sns.heatmap(numeric_data.corr(), annot=True, cmap='coolwarm'): This uses Seaborn's heatmap() function to create a visual representation of the correlation matrix:

- annot=True: Displays the correlation values on the heatmap.
- cmap='coolwarm': Uses a color scheme where red represents positive correlation, blue represents negative correlation, and lighter colors indicate weaker correlations.

In short, the code creates a heatmap that visually shows the correlations between all numeric features in your dataset. This is useful for identifying relationships between features and understanding how they might influence each other. Let me know if you have anything else!

Analysis Of Numeric Variables

```
# Select only numeric columns for mean imputation
numeric_data = data.select_dtypes(include=np.number)

# Impute NaNs in numeric columns only
data[numeric_data.columns] = data[numeric_data.columns].fillna(numeric_data.mean())
```

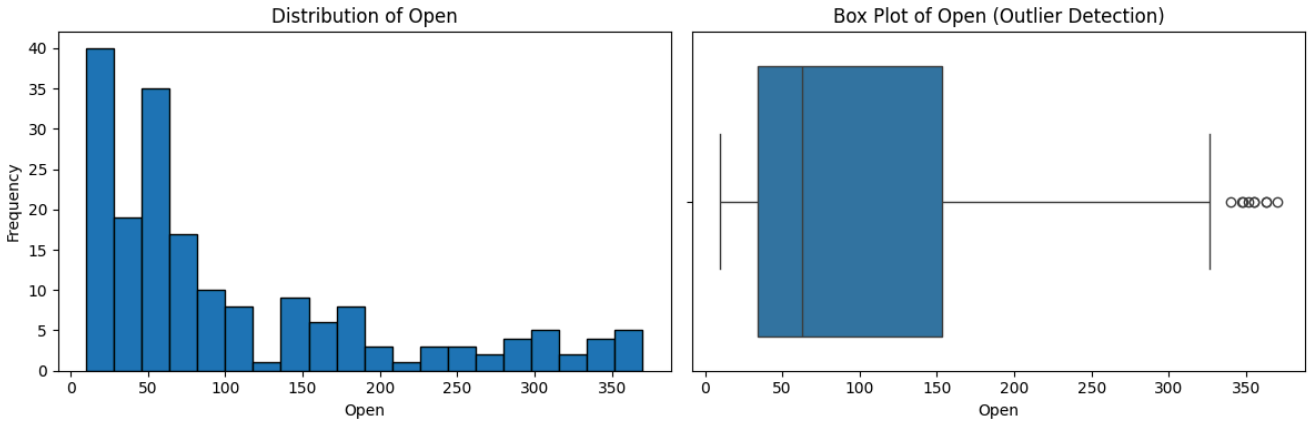
Okay, here's a short explanation of the code:

1. Select Numeric Columns: numeric_data = data.select_dtypes(include=np.number): This line identifies and selects only the columns in your dataset (data) that contain numeric data types.
2. Impute Missing Values with Mean: data[numeric_data.columns] = data[numeric_data.columns].fillna(numeric_data.mean()): This line fills any missing values (NaNs) in the selected numeric columns with the mean (average) value of each respective column.

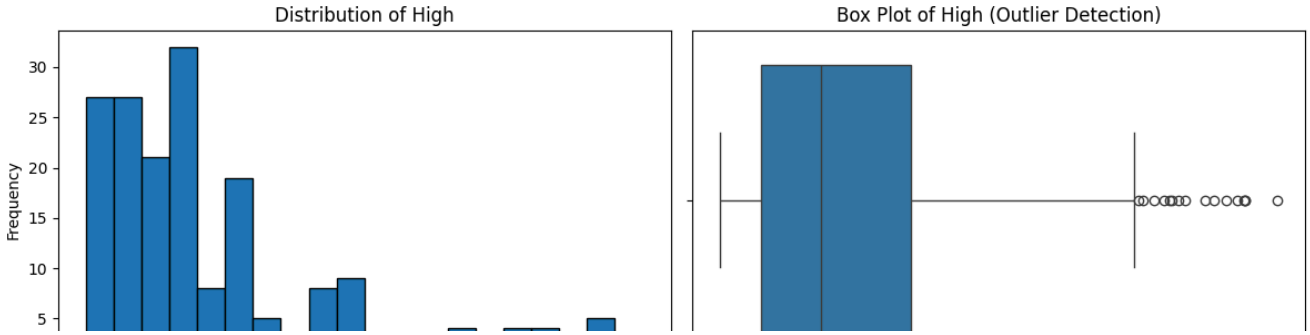
```
# Analysis of numeric variables with visualizations
numeric_features = data.select_dtypes(include=np.number).columns

for feature in numeric_features:
    print(f"Analysis of {feature}:")
    print(data[feature].describe())
    print("\n")

    # Create subplots for histogram and box plot
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))  # 1 row, 2 columns
```

```
# Histogram
axes[0].hist(data[feature], bins=20, edgecolor='black')
axes[0].set_title(f'Distribution of {feature}')
axes[0].set_xlabel(feature)
axes[0].set_ylabel('Frequency')

# Box plot
sns.boxplot(x=data[feature], ax=axes[1])  # Use Seaborn for box plot
axes[1].set_title(f'Box Plot of {feature} (Outlier Detection)')
axes[1].set_xlabel(feature)

plt.tight_layout()  # Adjust layout for better spacing
plt.show()
```

```
Analysis of Open:
count    185.000000
mean     105.541405
std       98.879850
min       10.000000
25%       33.800000
50%       62.980000
75%      153.000000
max      369.950000
Name: Open, dtype: float64
```




```
Analysis of High:
count    185.000000
mean      99.808506
std       84.091653
min       11.240000
25%       36.140000
50%       72.550000
75%      126.850000
max      350.000000
Name: High, dtype: float64
```



The code iterates through each numeric feature in your dataset (numeric_features) and performs the following:

1. Prints descriptive statistics: data[feature].describe() provides summary statistics (like mean, min, max, etc.) for the current feature.
2. Creates visualizations:

- Histogram: Shows the distribution of the feature's values (how often each value appears).
- Box plot: Visualizes the distribution and identifies potential outliers (data points that are significantly different from the rest).

In short, the code helps you understand the characteristics of each numeric feature in your dataset by displaying summary statistics and creating visual representations of their distributions and potential outliers. This analysis is important for understanding your data before applying machine learning models. Let me know if you have any other questions!

Feature Engineering

Feature Encoding

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline


# 1. Define categorical and numerical features
categorical_features = data.select_dtypes(include=['object']).columns
numerical_features = data.select_dtypes(include=np.number).columns

# 2. Create transformers for numerical and categorical features
numerical_transformer = Pipeline(steps=[
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder(sparse_output=False, handle_unknown='ignore'))
])

# 3. Create a ColumnTransformer to apply transformers to respective columns
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ])

# 4. Fit and transform the data
data_encoded = preprocessor.fit_transform(data)

#data_encoded is now a NumPy array. If you want to convert back to DataFrame:

# Get feature names after encoding
num_feature_names = numerical_features
cat_feature_names = preprocessor.named_transformers_['cat']['onehot'].get_feature_names_out(categorical_features)
all_feature_names = np.concatenate([num_feature_names, cat_feature_names])

# Create DataFrame
encoded_data = pd.DataFrame(data_encoded, columns=all_feature_names)
```

Okay, here's a short explanation of the feature encoding code:

What it does : The code transforms categorical features (like 'city', 'color') into numerical representations that machine learning models can understand. It also scales numerical features to a standard range.

Why it's important : Categorical Features: Many machine learning algorithms cannot directly work with categorical data. Encoding converts these features into numerical form. Scaling: Scaling ensures that features with large values don't disproportionately influence the model. **How it works :**

1. Identify Features: It finds which columns are categorical and which are numerical.
2. Create Transformers: It sets up tools (StandardScaler, OneHotEncoder) to handle each type of feature.
3. Apply Transformers: It uses ColumnTransformer to apply the correct tool to each column.
4. Encoded Data: You get a new dataset with encoded features, ready for machine learning.

In short, this code makes your data suitable for machine learning by converting categorical features into numbers and scaling numerical features. Let me know if you'd like more details on any specific part!

Create New Feature if require

```
# --- Create New Features (if required) ---

# 1. Example: Create a feature 'Open-Close Difference'
encoded_data['Open-Close Difference'] = encoded_data['Open'] - encoded_data['Close']

# 2. Example: Create a feature based on a condition (using np.where)
encoded_data['High_Low_Ratio'] = np.where(encoded_data['Low'] != 0, encoded_data['High'] / encoded_data['Low'], 0)


# ... (rest of your code) ...
```

1. Place After Encoding: Add this section after your feature encoding code.

2. Add Your Feature Logic: Replace the example feature creations with your own logic for creating new features. You can use mathematical operations, conditional statements, aggregations, etc.
3. Continue with Modeling: Use the updated DataFrame (encoded_data) with the new features for training and evaluating your models.

Removing Multicollinearity

```
# --- Remove Multicollinearity (if required) ---
import pandas as pd
import numpy as np
from statsmodels.stats.outliers_influence import variance_inflation_factor

# 1. Calculate VIF for each feature
def calculate_vif(df):
    vif_data = pd.DataFrame()
    vif_data["feature"] = df.columns
    vif_data["VIF"] = [variance_inflation_factor(df.values, i) for i in range(len(df.columns))]
    return vif_data

vif_df = calculate_vif(encoded_data[numerical_features])  # Apply to numerical features

# 2. Identify and remove features with high VIF
features_to_remove = vif_df[vif_df["VIF"] > 10]["feature"].tolist() # Remove features with VIF > threshold (e.g., 10)

encoded_data = encoded_data.drop(columns=features_to_remove)

# Update numerical_features list after removing features
numerical_features = [feature for feature in numerical_features if feature not in features_to_remove]

# ... (rest of your code) ...
```

How to Use:

1. Place After Feature Engineering: Add this section after creating new features (if you did) and before model training.
2. Threshold: Adjust the VIF threshold (e.g., 10) based on your data and analysis.
3. Iteration: You might need to iterate and recalculate VIF after removing features to ensure that multicollinearity is effectively addressed.

Important Considerations:

- VIF Threshold: The choice of VIF threshold can be subjective and depends on the specific dataset. Values above 5 or 10 are often considered indicators of high multicollinearity.
- Alternative Methods: There are other techniques to handle multicollinearity, such as principal component analysis (PCA) or ridge regression.
- Domain Knowledge: Domain expertise can help in selecting which features to remove when faced with highly correlated features.

Data Preprocessing

```
# Extract features and target
X = data[['Open', 'High', 'Low']]
y = data['Close']

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

Okay, here's a concise explanation of why those steps are used:

1. Feature/Target Extraction: Isolates the predictive variables (features) and the variable to be predicted (target).
2. Feature Scaling: Standardizes the features to a similar range, preventing features with larger values from dominating the model.
3. Train-Test Split: Divides the data into training and testing sets, allowing you to evaluate the model's performance on unseen data and prevent overfitting.

In essence, these steps prepare your data for machine learning by organizing, scaling, and splitting it to build and evaluate a reliable predictive model. I hope this shorter explanation is helpful!

Model Implementation

Model Training and Evaluation

Random Forest Regressor

```
# Train Random Forest
rf_model = RandomForestRegressor(random_state=42)
rf_model.fit(X_train, y_train)

# Predict and evaluate
rf_preds = rf_model.predict(X_test)
print("Random Forest Metrics:")
print("MAE:", mean_absolute_error(y_test, rf_preds))
print("MSE:", mean_squared_error(y_test, rf_preds))
print("RMSE:", np.sqrt(mean_squared_error(y_test, rf_preds)))
print("R^2:", r2_score(y_test, rf_preds))
```

```
Random Forest Metrics:
MAE: 8.72842162162161
MSE: 190.73562156054064
RMSE: 13.810706772665208
R^2: 0.9788982033612884
```

```
# Feature Importance for Random Forest
rf_importances = rf_model.feature_importances_
feature_names = ['Open', 'High', 'Low']  # Assuming these are your features
rf_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': rf_importances})
rf_importance_df = rf_importance_df.sort_values(by='Importance', ascending=False)

print("\nRandom Forest Feature Importance:")
print(rf_importance_df)

plt.figure(figsize=(8, 6))
plt.bar(rf_importance_df['Feature'], rf_importance_df['Importance'])
plt.title('Random Forest Feature Importance')
plt.xlabel('Feature')
```